Polarapt HCM Real-Time Data Platform: Hackathon Report

Muhammad Danish · Oct 25, 2026

Overview

This project is a real-time data pipeline on AWS for Polarapt HCM. It collects three public HR data feeds every minute, saves them in S3, and sends each feed to its own queue, processor, and final database.

The flow in one line: **three sources → Lambda (every minute) → S3 raw folder → SNS → router Lambda → FIFO SNS topic → three SQS FIFO queues → three processors → three destinations.**

I worked in the AWS region us-east-1. The S3 bucket is `data-hackathon-smit-danish17`. As my teacher advised, I finished two sources end to end first (FX and Benchmarks), then added the third (Jobs).

Data sources

The platform reads three public sources. Each one has its own Lambda, queue, and final store.

Source	What it gives	How it is read	Lambda name	Final store
USAJOBS	US federal job postings (up to 500 per call) with title, organization, location, salary, open and close dates	Search API with an API key	`lambda_jobs_ingest`	Snowflake

Source	What it gives	How it is read	Lambda name	Final store
BLS Highest Paying Occupations	Top 10 occupations by median annual pay	Web page scraping, no key needed	`lambda_benchmarks_scraper`	S3 processed zone (CSV)
Open Exchange Rates	Live rates for USD against PKR, SAR, AED, GBP, EUR, INR	API with an App ID	`lambda_fx_ingest`	SQL Server

Task 1: Data acquisition

Three Lambda functions fetch the data, and three Amazon EventBridge rules start them every minute. Each Lambda saves its result in S3.

- **Rules:** `fx-ingest-every-minute`, `benchmarks-scrape-every-minute`, and `jobs-ingest-every-minute`. Each one runs its Lambda at a rate of 1 minute.
- **File path:** `raw/<source>/YYYY/MM/DD/HHMM.json`, for example `raw/fx/2026/10/25/0545.json`.
- **Contract:** every file carries a small contract with the timestamp, source name, record count, and response status. It is saved in two places: inside the file, and as S3 object metadata.
- **Why two places:** the router can read the metadata without downloading the file, which is fast and cheap. The file also describes itself.

Two extra details: the BLS scraper uses Python's built-in `html.parser` and `urllib`, so it needs no extra library. The contract of the benchmarks file also has a `live_fetch` flag (see the problems section).

Processors

Each queue has its own processor, and EventBridge schedules start them. Each processor reads messages from its queue, loads the file from S3, does its work, and then deletes the message. If a processor fails, the message stays in the queue, so no data is lost.

Source	Processor	What it does	Destination
Benchmarks	`lambda_benchmarks_processor`	Ranks the occupations by median pay and writes a CSV	S3 `processed/benchmarks/...csv`
FX	`lambda_fx_processor` (with a `pymssql` layer)	Creates the database and table if missing, then inserts one row per currency	SQL Server on Amazon RDS, table `dbo.fx_rates`
Jobs	`lambda_jobs_processor` (with a Snowflake connector layer)	Flattens each posting into one row and inserts it	Snowflake table `HCM_DB.RAW.JOB_POSTINGS`

The FX and Jobs processors also check the source file name before loading. This stops the same file from being loaded twice.

Services and tools used

Service or tool	Why I used it
AWS Lambda (Python 3.12)	Runs the ingest, router, and processor code without servers
Amazon EventBridge	Starts the Lambdas on a schedule, every minute
Amazon S3	Stores raw files (`raw/`) and processed files (`processed/`)
Amazon SNS (standard topic `hcm-raw-events`)	Receives the S3 "object created" event, because S3 can only publish to a standard topic
Amazon SNS FIFO topic `hcm-routed-events.fifo`	Filters messages by the `source` attribute and fans them out

Service or tool	Why I used it
Amazon SQS FIFO queues	`jobs-queue.fifo`, `benchmarks-queue.fifo`, `fx-queue.fifo`: one buffer per source
Amazon RDS (SQL Server Express)	Holds the FX rates table
Snowflake	Warehouse for job postings (database `HCM_DB`, warehouse `HCM_WH`)
AWS IAM	Role `hcm-lambda-role` gives the Lambdas access
Lambda layers	`pymssql-layer` and `snowflake-layer`, built in AWS CloudShell
CloudWatch Logs	Shows errors and run details

Problems and fixes

Problem	What happened	Fix
BLS blocked the scraper	The BLS website returned HTTP 403 to requests from AWS, even with browser-like headers	The Lambda tries a live fetch first. If it fails, it saves a snapshot of the top 10 occupations, and the contract records `live_fetch` and `response_status` so everyone knows where the data came from
S3 cannot publish to a FIFO topic	S3 event notifications only work with a standard SNS topic, and they do not filter on our custom metadata	A router Lambda reads the metadata with `head_object` and republishes to the FIFO topic with a `source` attribute
Open Exchange Rates free limit	The free plan allows 1,000 requests a month, and one request per minute uses that in about 16 hours	The FX rule is enabled only for tests and demos, then disabled

Problem	What happened	Fix
Extra libraries in Lambda	`pymssql` and the Snowflake connector are not included in Lambda	Built them as layers in CloudShell. The Snowflake layer was uploaded through S3 because of its size
Snowflake login failed with a 404 error	I had used only the account locator, so the Snowflake URL did not exist	Used the full account identifier in the form `organization-account`
Wrong region in a command	I typed `REGION` instead of `us-east-1` when publishing a layer	Ran the command again with the real region

Differences from the brief, cost, and next steps

Differences from the brief

- The brief asks for a Glue job for the Jobs source. I used a Lambda function (`lambda_jobs_processor`) with a Snowflake layer, because my account is on a free plan. The logic is the same.
- The processed files go to a `processed/` folder in the same bucket, not a separate bucket.
- The BLS fallback snapshot is an addition for the 403 problem.

Cost and safety

- The IAM role has broad access (S3, SNS, SQS full access) and the SQL Server port 1433 is open to the internet. This is fine for a hackathon but not for production.
- After the demo: disable all EventBridge rules, stop or delete the RDS instance, and let the Snowflake warehouse auto-suspend (it is set to 60 seconds).

What I would improve

- Give each role only the access it needs, and put the Lambdas and RDS in a private network.
- Add a dead-letter queue to every SQS queue for messages that keep failing.
- Move the passwords and API keys from environment variables to AWS Secrets Manager.
- Add alarms in CloudWatch for failed Lambda runs and growing queues.